

Guida operativa

AI on-premise per laboratorio ITS

Setup di Docker Compose, Ollama, Open WebUI, RAG locale e n8n

Obiettivo	Allestire un ambiente locale per chat AI, test RAG e workflow di orchestrazione.
Stack	PostgreSQL, n8n, Ollama, Open WebUI, opzionale Botpress Cloud.
Uso consigliato	Didattica ITS, PoC interni, demo tecniche e primi casi d'uso aziendali.

1. Architettura consigliata

Per un laboratorio ITS il setup più equilibrato è il seguente:

- Ollama in Docker come motore LLM locale.
- Open WebUI in Docker come interfaccia web per chat e RAG.
- n8n in Docker come livello di orchestrazione e automazione.
- PostgreSQL come database di persistenza per n8n.
- Botpress Cloud facoltativo, da collegare in un secondo momento tramite webhook n8n.

Principio operativo: prima si mette in piedi la base tecnica locale (Ollama, Open WebUI, n8n), poi si introducono RAG, workflow e integrazioni cloud.

2. Prerequisiti

- Ubuntu o altra macchina Linux con Docker e Docker Compose già funzionanti.
- Accesso alla cartella del progetto contenente docker-compose.yaml e file .env.
- Porte disponibili: 5432 per PostgreSQL, 5678 per n8n, 11434 per Ollama, 3101 per Open WebUI.
- Almeno 16 GB RAM consigliati per una demo stabile con modello da 4B.
- Connessione internet iniziale per scaricare immagini Docker e modelli Ollama.

3. File .env consigliato

Aggiorna il file .env con:

```
# Porta per n8n
N8N_EXPOSED_PORT=5678
N8N_HOST=localhost
N8N_WEBHOOK_URL=http://localhost:5678/
```

```
# Porta per Ollama
OLLAMA_EXPOSED_PORT=11434
```

```
# Porta per Openwebui
OPENWEBUI_EXPOSED_PORT=3101
```

Perché Open WebUI usa la porta 3101? Per evitare conflitti con il range 3000-3099 già riservato ai servizi Node/Vite nel container dev.

4. Servizi Docker Compose

Modifica il tuo docker-compose.yaml ed aggiungi dopo Dataiku:

```
# =====
# n8n
# =====

n8n:
  image: docker.n8n.io/n8nio/n8n:latest
  container_name: ${COMPOSE_PROJECT_NAME}_n8n
  restart: unless-stopped
  depends_on:
    - postgresql
  ports:
    - "${N8N_EXPOSED_PORT:-5678}:5678"
  environment:
    - DB_TYPE=postgresdb
    - DB_POSTGRESDB_HOST=postgresql
    - DB_POSTGRESDB_PORT=5432
    - DB_POSTGRESDB_DATABASE=n8n
    - DB_POSTGRESDB_USER=postgres
    - DB_POSTGRESDB_PASSWORD=postgres
    - N8N_HOST=${N8N_HOST:-localhost}
    - N8N_PORT=5678
```

```

- N8N_PROTOCOL=http
- WEBHOOK_URL=${N8N_WEBHOOK_URL:-http://localhost:5678/}
- GENERIC_TIMEZONE=Europe/Rome
- TZ=Europe/Rome
volumes:
- sw_development_config_n8n:/home/node/.n8n
- ${USER_BASE_FOLDER}:/shared
networks:
- network

# =====
# ollama
# =====

```

```

ollama:
  image: ollama/ollama:latest
  container_name: ${COMPOSE_PROJECT_NAME}_ollama
  restart: unless-stopped
  ports:
    - "${OLLAMA_EXPOSED_PORT:-11434}:11434"
  volumes:
    - sw_development_config_ollama:/root/.ollama
    - ${USER_BASE_FOLDER}:/home/
  networks:
    - network

```

```

# =====
# openwebui
# =====
openwebui:
  image: ghcr.io/open-webui/open-webui:main
  container_name: ${COMPOSE_PROJECT_NAME}_openwebui
  restart: unless-stopped
  depends_on:
    - ollama
  ports:
    - "${OPENWEBUI_EXPOSED_PORT:-3101}:8080"
  environment:
    - OLLAMA_BASE_URL=http://ollama:11434
    - RAG_EMBEDDING_ENGINE=ollama
    - RAG_EMBEDDING_MODEL=embeddinggemma
    - RAG_TOP_K=4
    - CHUNK_SIZE=1000
    - CHUNK_OVERLAP=100
  volumes:
    - sw_development_config_openwebui:/app/backend/data
  networks:
    - network

```

Poi aggiorna la sezione volumes:

```
volumes:  
  sw_development_config_postgresql:  
  sw_development_config_pgadmin:  
  sw_development_config_dataiku:  
  sw_development_config_n8n:  
  sw_development_config_ollama:  
  sw_development_config_openwebui:
```

Salva docker-compose

Questa impostazione è in linea con la documentazione n8n per Docker Compose e con quella Ollama per Docker: n8n legge la config da environment variables, mentre Ollama espone l'API sulla porta 11434 e persiste i modelli in un volume dedicato

5. Sequenza corretta di installazione e avvio

1. Passo 1 - Avvia PostgreSQL

```
docker compose up -d postgresql
```

2. Passo 2 - Crea il database n8n

```
docker exec -it its_postgresql psql -U postgres -c "CREATE DATABASE n8n;"
```

3. Passo 3 - Avvia Ollama e Open WebUI

```
docker compose up -d ollama openwebui
```

4. Passo 5 - Verifica lo stato dei container

```
docker compose ps
```

Ordine corretto: il database n8n va creato prima di avviare n8n, altrimenti il container può andare in crash o restare in loop di restart.

6. Test di verifica servizio per servizio

Servizio	Comando di test	Risultato atteso	URL
PostgreSQL	<code>docker exec -it its_postgresql psql -U postgres -l</code>	Lista dei DB, compreso n8n	—
Ollama	<code>curl http://localhost:11434/api/tags</code>	JSON con i modelli installati	localhost:11434
Open WebUI	Aprire il browser	Interfaccia web disponibile	localhost:3101

6.1

Test Ollama: scarica un modello, testa e prova una generazione.

Scarica un modello

```
docker exec -it its_ollama ollama pull gemma3:4b
```

Poi ricontrolla:

```
curl http://localhost:11434/api/tags
```

Adesso dovresti vedere gemma3:4b nella lista. L'API tags serve proprio a elencare i modelli disponibili.

6.2— prova una generazione

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3:4b",
  "prompt": "Spiega in 3 righe cos è un workflow",
  "stream": false
}'
```

Se tutto è corretto, ricevi un JSON con un campo di risposta generata. L'endpoint POST /api/generate è uno degli endpoint base ufficiali di Ollama.

6.3 CLI nel container

```
docker exec -it its_ollama ollama run gemma3:4b
```

Usi il client CLI di Ollama dentro il container

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3:4b",
  "prompt": "Spiega in 3 righe cos è un workflow",
```

```
"stream": false
}'
```

7. Open WebUI e RAG locale

- Primo accesso: creare l'utente amministratore locale di Open WebUI.
- Modello chat da usare in conversazione: gemma3:4b.
- Modello embeddings per RAG: embeddinggemma.
- Schermata corretta per verificare la configurazione RAG: Admin → Impostazioni → Documents.
- Valori consigliati: Chunk Size 1000, Chunk Overlap 100, embedding engine Ollama.

Distinzione fondamentale: embeddinggemma non è un modello chat. Serve per creare vettori e retrieval. Per la conversazione usare sempre gemma3:4b o un altro modello generativo.

8. Creazione di una knowledge persistente

1. Aprire Open WebUI e andare in Workspace → Conoscenza.
2. Creare una nuova base di conoscenza con nome e descrizione.
3. Dopo la creazione, caricare uno o più file PDF, TXT o Markdown.
4. Attendere l'indicizzazione: il sistema estrae il testo, lo divide in chunk e crea gli embeddings.
5. Aprire una nuova chat con gemma3:4b, usare + → Attach Knowledge e selezionare la knowledge creata.

Cosa succede durante il caricamento? Open WebUI crea i chunk del documento, genera embeddings con il modello impostato e li salva nella knowledge base. Da quel momento il contenuto diventa richiamabile nelle chat future senza dover ricaricare il file ogni volta.

Esercizio 1 - Verifica base di Ollama

- Provare una chiamata via API con curl.
- Provare la stessa richiesta via CLI usando docker exec nel container Ollama.
- Confrontare i due metodi e descriverne le differenze.

Esercizio 2 - Prima RAG persistente

- Creare una knowledge
- Caricare un PDF di 2-5 pagine.
- Agganciare la knowledge a una chat.
- Fare tre domande puntuali sul contenuto.

Esercizio 3 - Verifica retrieval vs conoscenza generale

- Fare una domanda su un dettaglio presente nel documento.
- Fare una seconda domanda su un'informazione non presente nel documento.
- Spiegare la differenza tra risposta grounded e risposta generica.

9. n8n

1. Passo 4 - Avvia n8n

```
docker compose up -d n8n
```

2. Passo 5 - Verifica lo stato dei container

```
docker compose ps
```

Test di verifica

Servizio	Comando di test	Risultato atteso	URL
n8n	Aprire il browser	Interfaccia n8n disponibile	localhost:5678